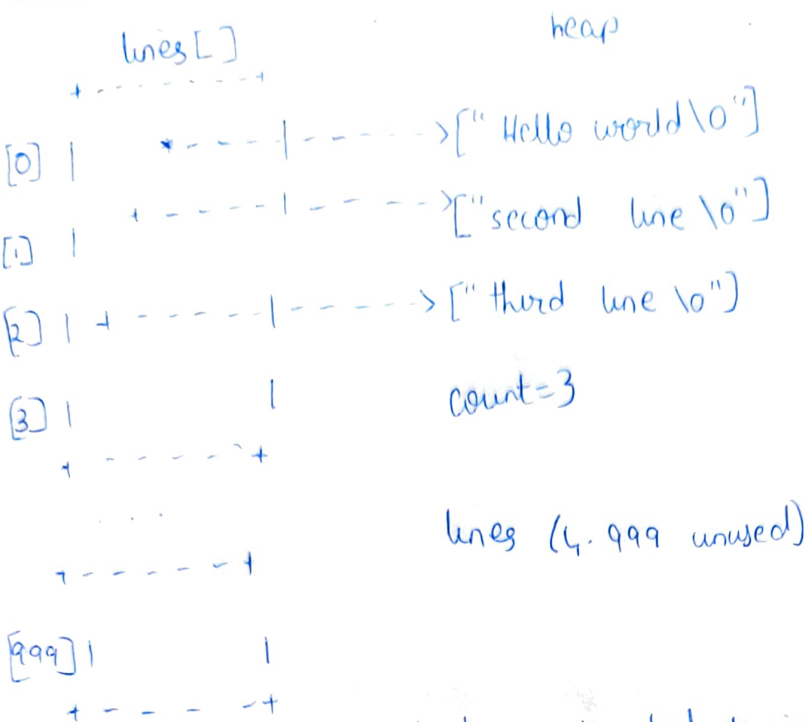


Data Structure :-

→ An array of pointer to heap-allocated strings

Struct :-

```
#define MAX_LINES 1000
#define MAX_LEN 512

typedef struct {
    char* lines[MAX_LINES];
    int count;
```

```
} Document;
```

```
static Document doc; backup;
```

→ We have chosen a structure of fixed array of 1000 pointers each pointing to a malloc'd string

Command Set

command	meaning	example
i<n><text>	Insert text as line n, shift the text down	i 3 hello world
d<n>	Delete line n, shift the text up	d 3
p	print the whole doc with line numbers	p
s<file>	save the document to a file	s notes.txt
l<file>	load a file replacing the doc	l notes.txt
f<word>	find which lines contain a word	f error
r<n><old>	replace first old on line n	r 2 cat dog
h	show help text	h
q	Quit	q

~~Functions List:-~~

~~* helpers *~~

~~void stamp~~

Function List

/* helpers */

void strip(char *s);

int valid(int n),

char *dup_str(const char *s),

void doc_free(Document *d),

void snapshot(void),

/* core */

int cmd_insert(int n, const char *text);

int cmd_delete(int n);

int cmd_print(void),

int cmd_save(const char *f);

int cmd_load(const char *f);

/* bonus */

void cmd_search(const char *word);

int cmd_replace(int n, const char *old, const char *new);

void cmd_status(void);

int cmd_undo(void);

void print_help(void);

④ insert (hand written)

int cmd_insert(int n, char *text)

{
if (doc.count >= maxLine || n < 1 || n > doc.count+1)
return 0;

char *copy = dup_str(text);

if (!copy) return 0;

for (int i = doc.count; i > n-1; i++)

doc.line[i] = doc.line[i-1];

doc.line[n-1] = copy;

doc.count++;

return 1;

}

Draw this beside it - insert "X" at line 3,

count = 3.

before: [A][B][C][C] i=3, copy index 2 → 3

step 2: [A][B][B][C] i=2, copy index 1 → 2

step 3: [A][X][B][C] (look for (i) 1) false

write copy at
index 1

(count+1)

Page 5 - delete 1

incl incl, delete (w n)

```
if (!valid(n)) return 0;
```

```
free (doc->lines[n-1]);
```

```
for (int i = n-1; i < doc->count-1; i++)
```

```
doc->lines[i] = doc->lines[i+1];
```

```
doc->count--;
```

```
return 1;
```

```
}
```

delete line, count = 4;

before: [A][X][B][C] count 4

1 Free this pointer first

Step 1: [A][B][B][C] i = 1, copy index 2 → 1

Step 2: [A][B][C][C] i = 1, copy index 3 → 2

loop stop (i < 3 false)

Step 3: [A][B][C] count 3 - slot 3 is now unused

Page 6:

void _init(void)

{

if (!doc.count) { printf("Document is empty\n");

return;

for (int i = 0; i < doc.count; i++)

printf("%s\n", doc.doc[i]);

}

Edge cases

Situation

iD text -line number 1

999 text on a 3-line doc

14 text on a 3-line doc

15 on a 3-line doc

11 on a 1-line doc

on an empty document

1 missing.txt

5 bad/path.txt

Document already at

1000 lines

check + D at the prompt

What the editor does

Reject, error message,
document unchanged

Rejected only count + 1 is
valid

Allowed - this is append

Rejected, error message
works, document becomes
empty

prints (doc is empty)

open returns Null

proper - - -

Insert rejected

gets returns Null
editor exists cleanly

Team

→ Trivikram Kaligoi

→ Sathakam Ram

→ Anand Kumar

→

→

→

→

→

→

→

→

→

→

→

→

→

→

→

→

→

→

→

→

→

19/06 2020

19/06 2020

C line_editor.c

You, 25 minutes ago | 1 author (You)

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #define MAX_LINES 1000
6  #define MAX_LEN 512
7
8  typedef struct {
9      char *lines[MAX_LINES];
10     int count;
11 } Document;
12
13 static Document doc, backup;
14 static int has_backup = 0;
15
16 void strip(char *s) {
17     size_t n = strlen(s);
18     while (n && (s[n-1] == '\n' || s[n-1] == '\r')) s[--n] = '\0';
19 }
20
21 int valid(int n) { return n >= 1 && n <= doc.count; }
22
23 char *dup_str(const char *s) {
24     char *p = malloc(strlen(s) + 1);
25     if (p) strcpy(p, s);
26     return p;
27 }
28
29 void doc_free(Document *d) {
30     for (int i = 0; i < d->count; i++) free(d->lines[i]);
31     d->count = 0;
32 }
```

```
29 void doc_free(Document *d) {
30     d->count = 0;
31 }
32
33
34 void snapshot(void) {
35     doc_free(&backup);
36     for (int i = 0; i < doc.count; i++) backup.lines[i] = dup_str(doc.lines[i]);
37     backup.count = doc.count;
38     has_backup = 1;
39 }
40
41 int cmd_insert(int n, const char *text) {
42     if (doc.count >= MAX_LINES || n < 1 || n > doc.count + 1) return 0;
43     char *copy = dup_str(text);
44     if (!copy) return 0;
45     for (int i = doc.count; i > n - 1; i--) doc.lines[i] = doc.lines[i-1];
46     doc.lines[n-1] = copy;
47     doc.count++;
48     return 1;
49 }
50
51 int cmd_delete(int n) {
52     if (!valid(n)) return 0;
53     free(doc.lines[n-1]);
54     for (int i = n - 1; i < doc.count - 1; i++) doc.lines[i] = doc.lines[i+1];
55     doc.count--;
56     return 1;
57 }
58
59 void cmd_print(void) {
60     if (!doc.count) { printf("(document is empty)\n"); return; }
61     for (int i = 0; i < doc.count; i++) printf("%4d | %s\n", i+1, doc.lines[i]);
62 }
```

```

61     for (int i = 0; i < doc.count; i++) printf("%4d | %s\n", i+1, doc.lines[i]);
62 }
63
64 int cmd_save(const char *f) {
65     FILE *fp = fopen(f, "w");
66     if (!fp) return 0;
67     for (int i = 0; i < doc.count; i++) fprintf(fp, "%s\n", doc.lines[i]);
68     fclose(fp);
69     return 1;
70 }
71
72 int cmd_load(const char *f) {
73     FILE *fp = fopen(f, "r");
74     if (!fp) return 0;
75     doc_free(&doc);
76     char buf[MAX_LEN];
77     while (doc.count < MAX_LINES && fgets(buf, sizeof buf, fp)) {
78         strip(buf);
79         doc.lines[doc.count++] = dup_str(buf);
80     }
81     fclose(fp);
82     return 1;
83 }
84
85 void cmd_search(const char *word) {
86     int hits = 0;
87     for (int i = 0; i < doc.count; i++)
88         if (strstr(doc.lines[i], word)) { printf("%4d | %s\n", i+1, doc.lines[i]); hits++; }
89     printf(hits ? "%d line(s) matched.\n" : "Not found.\n", hits);
90 }
91
92 int cmd_replace(int n, const char *old, const char *new) {

```

```

92  int cmd_replace(int n, const char *old, const char *new) {
93      if (!valid(n)) return 0;
94      char *hit = strstr(doc.lines[n-1], old);
95      if (!hit) return 0;
96      char buf[MAX_LEN * 2];
97      int prefix = (int)(hit - doc.lines[n-1]);
98      sprintf(buf, ".*%s%s", prefix, doc.lines[n-1], new, hit + strlen(old));
99      free(doc.lines[n-1]);
100     doc.lines[n-1] = dup_str(buf);
101     printf("%4d | %s\n", n, doc.lines[n-1]);
102     return 1;
103 }
104
105 void cmd_stats(void) {
106     int words = 0, chars = 0;
107     for (int i = 0; i < doc.count; i++) {
108         chars += (int)strlen(doc.lines[i]);
109         for (int in = 0, j = 0; doc.lines[i][j]; j++) {
110             if (doc.lines[i][j] == ' ' || doc.lines[i][j] == '\t') in = 0;
111             else if (!in) { in = 1; words++; }
112         }
113     }
114     printf("Lines: %d   Words: %d   Characters: %d\n", doc.count, words, chars);
115 }
116
117 int cmd_undo(void) {
118     if (!has_backup) return 0;
119     Document t = doc; doc = backup; backup = t;
120     return 1;
121 }
122
123 void print_help(void) {
124     printf("\n  i <n> <text>          insert text as line n\n")

```

```

122
123 void print_help(void) {
124     printf("\n  i <n> <text>          insert text as line n\n"
125           "    d <n>              delete line n\n"
126           "    p                      print the document\n"
127           "    s <file>              save to file\n"
128           "    l <file>              load from file\n"
129           "    f <word>             find lines containing word\n"
130           "    r <n> <old> <new>    replace old with new on line n\n"
131           "    c                      line/word/character counts\n"
132           "    u                      undo last change\n"
133           "    h                      this help\n"
134           "    q                      quit\n\n");
135 }
136
137 int main(void) {
138     char in[MAX_LEN], cmd[16], a[MAX_LEN], b[MAX_LEN];
139     int n, off;
140
141     printf("Line Editor. Type 'h' for help, 'q' to quit.\n");
142
143     while (printf("> "), fflush(stdout), fgets(in, sizeof in, stdin)) {
144         strip(in);
145         if (sscanf(in, "%15s", cmd) != 1) continue;
146
147         if (!strcmp(cmd, "q")) break;
148         else if (!strcmp(cmd, "h")) print_help();
149         else if (!strcmp(cmd, "p")) cmd_print();
150         else if (!strcmp(cmd, "c")) cmd_stats();
151         else if (!strcmp(cmd, "u")) printf(cmd_undo() ? "Undone.\n" : "Nothing to undo.\n");
152         else if (!strcmp(cmd, "i")) {
153             off = 0;
154             if (sscanf(in, "%*s %d %n", &n, &off) == 1 && off > 0) {

```

```

153     off = 0;
154     if (sscanf(in, "%*s %d %n", &n, &off) == 1 && off > 0) {
155         snapshot();
156         if (!cmd_insert(n, in + off)) printf("Error: cannot insert at line %d.\n", n);
157     } else printf("Usage: i <lineno> <text>\n");
158 }
159 else if (!strcmp(cmd, "d")) {
160     if (sscanf(in, "%*s %d", &n) == 1) {
161         snapshot();
162         if (!cmd_delete(n)) printf("Error: no line %d.\n", n);
163     } else printf("Usage: d <lineno>\n");
164 }
165 else if (!strcmp(cmd, "s")) {
166     if (sscanf(in, "%*s %255s", a) == 1)
167         printf(cmd_save(a) ? "Saved.\n" : "Error: cannot write file.\n");
168     else printf("Usage: s <filename>\n");
169 }
170 else if (!strcmp(cmd, "l")) {
171     if (sscanf(in, "%*s %255s", a) == 1) {
172         snapshot();
173         printf(cmd_load(a) ? "Loaded.\n" : "Error: cannot read file.\n");
174     } else printf("Usage: l <filename>\n");
175 }
176 else if (!strcmp(cmd, "f")) {
177     off = 0;
178     sscanf(in, "%*s %n", &off);
179     if (off > 0 && in[off]) cmd_search(in + off);
180     else printf("Usage: f <word>\n");
181 }
182 else if (!strcmp(cmd, "r")) {
183     if (sscanf(in, "%*s %d %511s %511s", &n, a, b) == 3) {
184         snapshot();
185         if (!cmd_replace(n, a, b)) printf("Not found on line %d.\n", n);

```

```
181     }
182     else if (!strcmp(cmd, "r")) {
183         if (sscanf(in, "%*s %d %511s %511s", &n, a, b) == 3) {
184             snapshot();
185             if (!cmd_replace(n, a, b)) printf("Not found on line %d.\n", n);
186             } else printf("Usage: r <lineno> <old> <new>\n");
187         }
188     else printf("Unknown command '%s'. Type 'h' for help.\n", cmd);
189 }
190
191 doc_free(&doc);
192 doc_free(&backup);
193 printf("Goodbye.\n");
194 return 0;
195 }
```



Portfolio-building-activity

Public

Pin



main



1 Branch



0 Tags



Go to file



Add file



Code



saardhakram

Line editor: core insert, delete, display, save/load

2f1b188 · 27 minutes ago



4 Commits



README.md

pushed readme.md

last week



a.exe

Line editor: core insert, delete, display, save/load

27 minutes ago



activity.c

Line editor: core insert, delete, display, save/load

27 minutes ago



line_editor.c

Line editor: core insert, delete, display, save/load

27 minutes ago



notes.txt

Line editor: core insert, delete, display, save/load

27 minutes ago